Software Requirements Specification

Restaurant App MVP (Frontend Only)

React Native (Expo)

Version 1.0

Mobile App Development, Fall 2026

Prepared by: Bisma

Reg. No. 9436, BS Computer Science

Abasyn University, Islamabad Campus

October 2026

1. Introduction

1.1 Purpose

This document defines the requirements of the Restaurant App MVP, version 1.0. It is written for the course instructor (who evaluates it), for the developer (who implements it), and for any tester who needs to verify the app. It describes a React Native (Expo) mobile app that lets Customers browse a menu, search, build a cart, apply promo codes, book tables and place and track orders, and lets a Restaurant Manager handle orders, reservations and menu items. Version 1.0 is frontend only: all data is mock data held in React state and saved on the device with AsyncStorage.

1.2 Scope

In scope:

- Signup and login with a role selector (Customer or Manager), with form validation and role-based navigation.
- Menu browsing: at least 15 items in 4 categories (Starters, Mains, Desserts, Drinks), category chips, Daily Special badge, greyed-out unavailable items, pull to refresh.
- Search with debounce, recent searches, sort (price low to high, price high to low, name A to Z), back to top button and favourites.
- Cart with quantity stepper, special instructions per item, promo codes (WELCOME10 and FEAST20) and a live badge on the cart tab.
- Order summary with service charge (5%), sales tax (15%), discount and grand total; Dine-in (with table) or Takeaway (with pickup time) ordering.
- Table reservation with hourly slots from 12:00 to 22:00, availability checking, validation, confirmation modal and My Reservations with cancellation.
- Order tracking with step progress and elapsed time (automatic demo status changes at 10, 20 and 30 seconds).
- Profile screen with light and dark theme toggle and logout.
- Manager Dashboard (Manager only) with Incoming Orders, Reservations (accept or decline) and Menu Management (add item, edit price, toggle availability).

Out of scope:

- Real payment processing (card, wallet, cash on delivery handling).
- Push notifications.
- Any backend service, external API or database; data is not shared between devices.
- Delivery tracking, maps and GPS.
- State management libraries such as Redux (only React built-in hooks are allowed).

1.3 Definitions and Acronyms

Term	Definition
SRS	Software Requirements Specification: this document, which states what the system must do.
MVP	Minimum Viable Product: the smallest working version with the core features of the Restaurant App.
UML	Unified Modeling Language: standard diagrams (use case, class, sequence, state, component) used to design the system.
Hook	A React function (such as useState) that lets a function component use state and lifecycle features.

Term	Definition
Context	A React mechanism (createContext and useContext) for sharing data such as the logged-in user without passing props through every level.
Reducer	A pure function (state, action) that returns the next state; used with useReducer for the cart and orders.
Mock Data	Fake sample data (users, menu, tables) written in JavaScript files and used instead of a real database.
FlatList	A React Native component that renders long lists efficiently by drawing only the visible rows.
Debounce	Waiting until the user stops typing (400 ms) before running the search, to avoid work on every keystroke.
AsyncStorage	A key-value store on the phone used to keep data after the app closes.
Expo / Expo Go	A toolchain and a phone app used to run and test React Native projects without building native binaries.
React Navigation	The library that provides stack and bottom-tab navigation between screens.
Dine-in / Takeaway	Dine-in: the customer eats at a restaurant table. Takeaway: the customer collects food at a chosen pickup time.

2. Overall Description

2.1 User Roles

Role	Key Permissions and Goals
Customer	Sign up and log in; browse, search, sort and favourite menu items; manage a cart and apply promo codes; place Dine-in or Takeaway orders; track order status; reserve a table, view and cancel own reservations; switch theme; view profile. Goal: order and book quickly without queues. Cannot open the Manager Dashboard.
Restaurant Manager	Log in with a manager account; see and update the status of incoming orders; accept or decline reservations; add menu items, edit prices and toggle availability; all customer-facing screens including theme and profile. Goal: digitise the menu and manage orders and bookings efficiently.

Self-interview (client meeting). Interviewing myself as a customer, the features that would make me download the app are: seeing today's specials without calling the restaurant; knowing which dishes are finished before I choose; searching for a dish by name; adding notes like 'no onions'; applying a discount code before paying; booking a table for 7:00 PM and being told if no table is free; and watching my order move from Pending to Served so I know when to arrive. As a manager I want to stop phone-in orders, change prices and availability in one place, and approve bookings. These points are the source of the user stories below.

2.2 User Stories

ID	User Story	Source
US-01	As a Customer, I want to see the Daily Special items, so that I can pick the best dish quickly.	Self-interview
US-02	As a Customer, I want unavailable dishes to be greyed out, so that I do not order something that is finished.	Self-interview
US-03	As a Customer, I want to search the menu by name, so that I can find a dish without scrolling.	Self-interview
US-04	As a Customer, I want to add a note such as 'no onions' to a cart item, so that my food is prepared the way I like.	Self-interview
US-05	As a Customer, I want to apply a promo code, so that I pay less.	Self-interview
US-06	As a Customer, I want to reserve a table for a chosen time and party size, so that my seat is ready when I arrive.	Self-interview

ID	User Story	Source
US-07	As a Customer, I want to track my order status and elapsed time, so that I know when my food will be ready.	Self-interview
US-08	As a Customer, I want to switch to dark mode, so that the app is comfortable to use at night.	Self-interview
US-09	As a Customer, I want to sort the menu by price or name, so that I can compare options.	Self-interview
US-10	As a Restaurant Manager, I want to see incoming orders and change their status, so that customers always see the real progress.	Manager
US-11	As a Restaurant Manager, I want to accept or decline reservations, so that I can control table use.	Manager
US-12	As a Restaurant Manager, I want to add items, edit prices and toggle availability, so that the menu is always correct.	Manager
US-13	As a Restaurant Manager, I want the Manager Dashboard to be hidden from customers, so that only staff can change orders and the menu.	Manager

3. Functional Requirements

ID	Module	Requirement
FR-01	Authentication	The system shall switch the entry screen between Login mode and Signup mode.
FR-02	Authentication	The system shall require full name, email, password, confirm password and a role (Customer or Manager) for signup, and email and password for login.
FR-03	Authentication	The system shall validate on submit that the email format is valid, the password has at least 8 characters including at least one digit, and password and confirm password match.
FR-04	Authentication	The system shall clear the error of a field as soon as the user edits that field.
FR-05	Authentication	The system shall show an ActivityIndicator and disable the submit button while a request is in progress (1 second simulated delay).
FR-06	Authentication	The system shall open the Menu screen for a Customer and the Manager Dashboard for a Manager after successful login, and show an Alert with a clear message on failure.
FR-07	Authentication	The system shall keep the logged-in user in AuthContext and shall return to the Login screen with a reset navigation stack on logout.
FR-08	Menu	The system shall display at least 15 menu items in at least four categories (Starters, Mains, Desserts, Drinks) in a FlatList.
FR-09	Menu	The system shall show a loading indicator while the menu loads and an error message with a Retry button if loading fails.
FR-10	Menu	The system shall show a Daily Special badge on items whose isSpecial is true.
FR-11	Menu	The system shall grey out unavailable items and disable their Add to Cart button.
FR-12	Menu	The system shall filter the menu by the category chip the user selects and show the number of items in the screen header.
FR-13	Menu	The system shall reload the menu when the user pulls down to refresh.
FR-14	Search	The system shall apply the typed search text only after 400 ms without typing (debounce).
FR-15	Search	The system shall sort items by price low to high, price high to low or name A to Z.
FR-16	Search	The system shall keep the last five distinct search terms and show them as tappable suggestions when the search box is focused and empty.
FR-17	Search	The system shall show a friendly empty-state message when no item matches the search.
FR-18	Search	The system shall show a Back to top button after the user scrolls down 300 pixels.
FR-19	Cart	The system shall add an item to the cart from any menu card and show a live item count badge on the cart tab.
FR-20	Cart	The system shall increase or decrease item quantity and remove the item when its quantity reaches zero.

ID	Module	Requirement
FR-21	Cart	The system shall let the user enter a special instruction for each cart item.
FR-22	Cart	The system shall accept the promo codes WELCOME10 (10%) and FEAST20 (20%) and reject any other code with an error message.
FR-23	Cart	The system shall calculate subtotal, 5% service charge, 15% sales tax, promo discount and grand total on the Order Summary screen.
FR-24	Reservation	The system shall offer hourly time slots from 12:00 to 22:00 and disable a slot when no table with enough seats is free at that time.
FR-25	Reservation	The system shall reject a reservation whose date is in the past, whose party size is outside 1 to 12, whose phone does not match 03XX-XXXXXXX, or which starts less than one hour ahead.
FR-26	Reservation	The system shall show a confirmation modal summarising the booking before it is saved.
FR-27	Reservation	The system shall list the user's reservations and cancel one only after the user confirms an Alert.
FR-28	Orders	The system shall let the customer choose Dine-in (with a table) or Takeaway (with a pickup time) before placing an order.
FR-29	Orders	The system shall create an order with id, items, total, type, status Pending and a timestamp when the customer places it.
FR-30	Orders	The system shall automatically move an order from Pending to Preparing after 10 seconds, to Ready after 20 seconds and to Served after 30 seconds (demo timings), and show a step progress indicator with an elapsed time counter updated every second.
FR-31	Orders	The system shall allow an order to be cancelled only while its status is Pending.
FR-32	Manager Dashboard	The system shall show the Manager Dashboard tab only when user.role is manager.
FR-33	Manager Dashboard	The system shall let the manager change order status from the Incoming Orders tab.
FR-34	Manager Dashboard	The system shall let the manager accept or decline reservations from the Reservations tab.
FR-35	Manager Dashboard	The system shall let the manager add an item, edit a price and toggle availability from the Menu Management tab, and the change shall appear on the customer menu immediately through shared state.
FR-36	Profile and Theme	The system shall show the logged-in user's name, email and role on the Profile screen.
FR-37	Profile and Theme	The system shall toggle between light and dark palettes from a switch on the Profile screen and apply it to every screen immediately.

4. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Usability	Every screen shall use the same navigation pattern (bottom tabs plus stacks), large touch targets (minimum 44 px) and clear inline error messages. A new user shall be able to place an order in under 2 minutes without help.
NFR-02	Usability	Both light and dark themes shall keep text readable (sufficient contrast) on every screen.
NFR-03	Performance	A screen shall show its first content or a loading indicator within 1 second; the menu list (15+ items) shall scroll smoothly using FlatList with keyExtractor.
NFR-04	Performance	MenuItemCard shall be wrapped in React.memo with useCallback handlers so that toggling one favourite re-renders only that card; totals shall be computed with useMemo.
NFR-05	Responsiveness	Layouts shall adapt to small (360 px wide) and large phones and tablets using flex layouts and percentages, and shall respect safe areas (notch and home indicator).

ID	Category	Requirement
NFR-06	Maintainability	Source shall be organised under src with the folders components, screens, context, reducers, hooks, data, navigation; business logic shall live in hooks and reducers, not in screens.
NFR-07	Maintainability	Reusable components (MenuItemCard, CategoryChips, SearchBar, Loader) shall be reused instead of copied; colours shall come only from the theme file.
NFR-08	Data handling (no backend)	All data shall be mock JavaScript data loaded into React state. Orders, reservations and menu edits shall be saved to AsyncStorage with useEffect and reloaded at start-up, with a loading screen so the app never flashes empty data.
NFR-09	Data handling (no backend)	The app shall run without any backend, external API or state management library.

5. Client-Side Data Model (Mock Data)

Because no database is used, the following data sets are stored as local JavaScript files in src/data and held in React state (and AsyncStorage for orders, reservations and menu edits).

Data Set Name	Fields	Description
Users (src/data/users.js)	id, fullName, email, password, role ('customer' or 'manager')	Mock accounts for login. At least one customer and one manager. New signups are appended to the array in state.
Categories	id, name (Starters, Mains, Desserts, Drinks)	Used by the category chips and to group menu items.
MenuItems (src/data/menu.js)	id, name, description, price (number), category, image, isSpecial (boolean), isAvailable (boolean)	At least 15 items across the four categories. Loaded through a Promise that resolves after 1.5 seconds. Manager edits change this data in state.
Tables (mockTables)	id, number, seats	At least six tables with different seat counts (2, 4, 6, 8) used for availability checks.
Reservations (mockReservations)	id, userId, tableId, date, timeSlot, partySize, phone, status ('pending', 'accepted', 'declined', 'cancelled'), createdAt	Bookings. A slot is unavailable when every table with enough seats is booked at that date and time.
Orders	id, userId, items [{itemId, name, price, quantity, note}], subtotal, total, type ('dine-in' or 'takeaway'), tableId or pickupTime, status (Pending, Preparing, Ready, Served, Cancelled), timestamp	Created by the place-order flow into OrdersContext (useReducer) and persisted in AsyncStorage.
PromoCodes	code, discountPercent	Local object: WELCOME10 = 10, FEAST20 = 20.

6. UML Diagrams

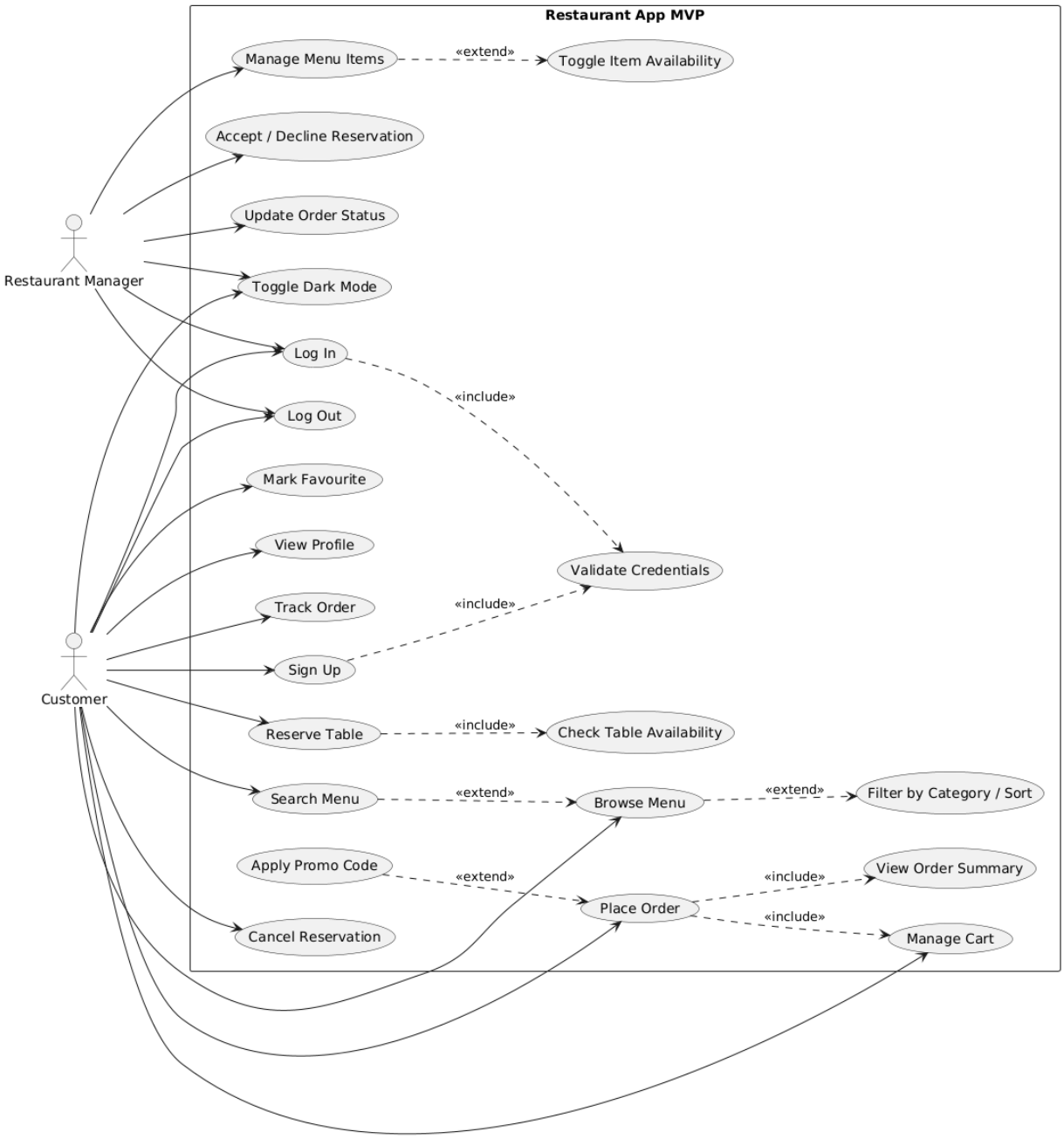


Figure 6.1: Use Case Diagram

The use case diagram shows the two actors, Customer and Restaurant Manager, and 22 use cases. Place Order includes View Order Summary and Manage Cart, Sign Up and Log In include Validate Credentials, and Reserve Table includes Check Table Availability. Apply Promo Code extends Place Order, Search Menu and Filter/Sort extend Browse Menu, and Toggle Item Availability extends Manage Menu Items. Both actors share Log In, Toggle Dark Mode and Log Out.

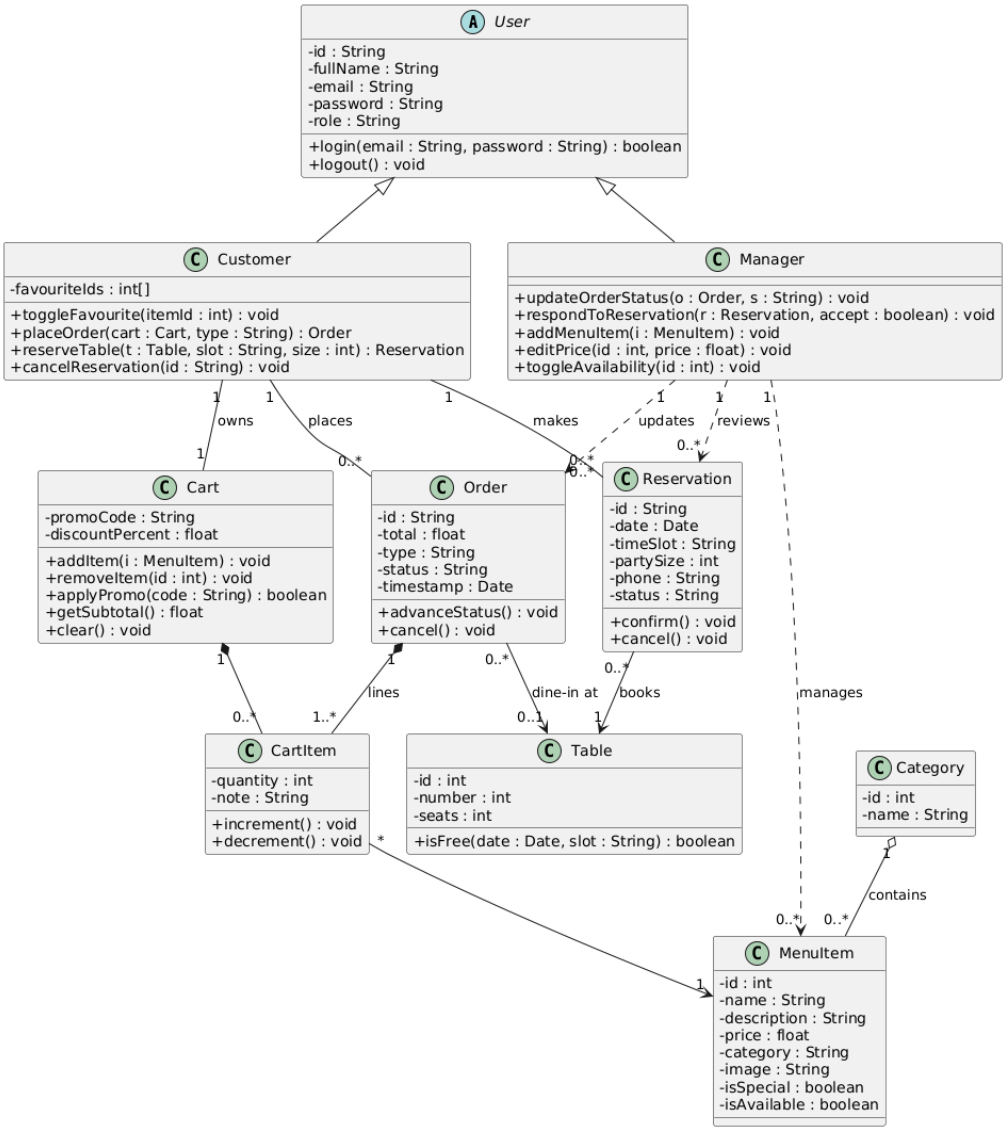


Figure 6.2: Class Diagram

The class diagram shows User as a parent class of Customer and Manager (inheritance). A Customer owns one Cart made of CartItems that point to MenuItems, places many Orders and makes many Reservations; each Reservation books one Table. Category groups MenuItems. Attributes show types and multiplicities are written on the lines. Managers update Orders, review Reservations and manage MenuItems.

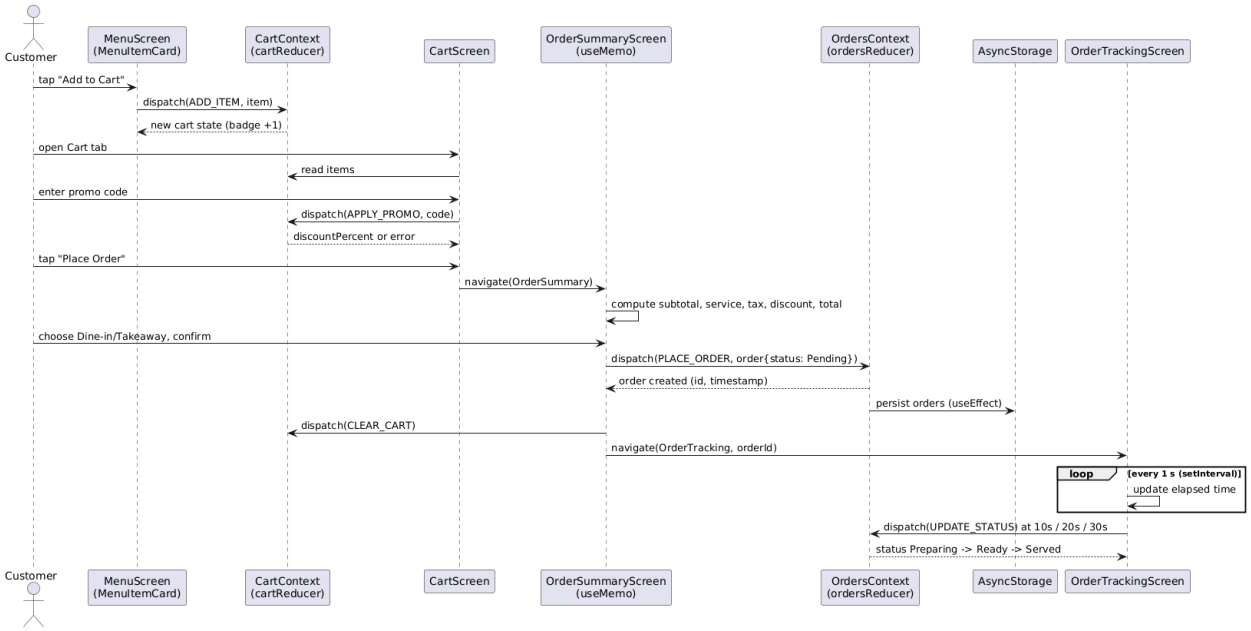


Figure 6.3: Sequence Diagram - Customer places an order

The sequence diagram follows the scenario from tapping Add to Cart on a menu card to seeing the Order Tracking screen. The cart reducer handles ADD_ITEM and APPLY_PROMO, the Order Summary screen computes totals with useMemo, OrdersContext creates the order with status Pending and saves it to AsyncStorage, the cart is cleared, and Order Tracking runs a one second timer and advances the status.

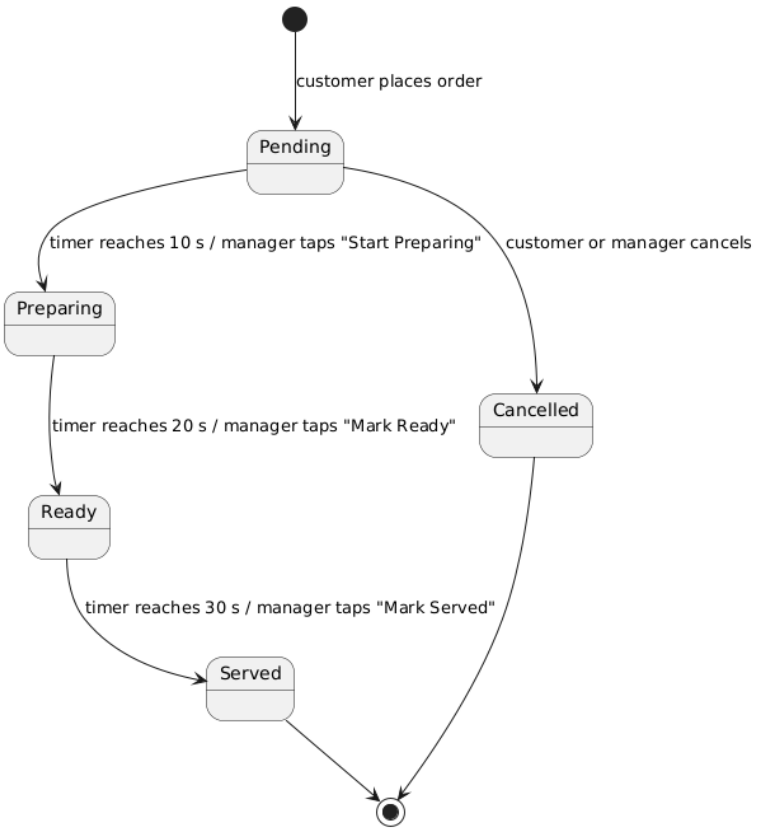


Figure 6.4: State Machine Diagram - Order lifecycle

An order starts in Pending when placed. It moves to Preparing, then Ready, then Served, either automatically (demo timers at 10, 20 and 30 seconds) or when the manager changes it. Cancellation is allowed only from Pending. Served and Cancelled are final states.

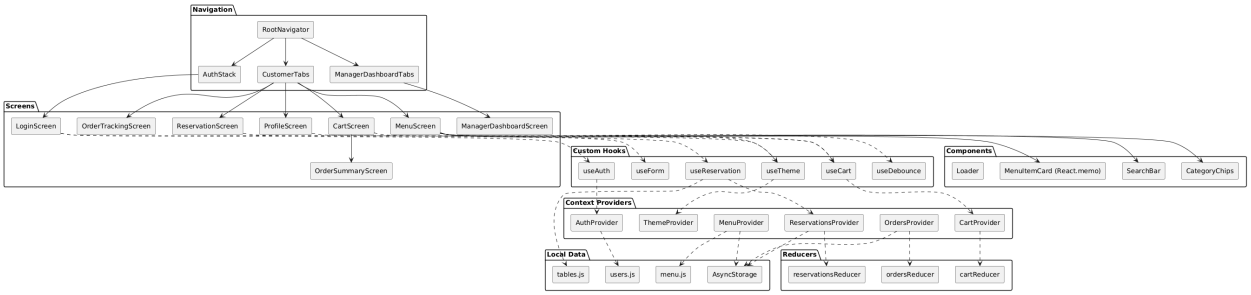


Figure 6.5: Component Diagram - React Native app

The component diagram shows the navigators, the screens, the reusable components, the context providers with their reducers, the custom hooks and the data layer. Screens depend on hooks, hooks depend on providers, providers depend on reducers and on local data or AsyncStorage.

7. MVP Frontend Development (React Native)

7.1 Login and Signup Screen

Purpose	Let a user create an account or log in and route them by role.
UI elements	Mode switch (Login/Signup), TextInputs for full name, email, password, confirm password, role selector (Customer/Manager), show-password toggle, error text under each field, submit button with ActivityIndicator, Alert on failure.
Navigation (entry / exit)	Entry: app start when no user is logged in; also after logout. Exit: Customer goes to Menu tab; Manager goes to Manager Dashboard.
Data used	src/data/users.js (mock users array).
Hooks and why	useState (mode, values, errors, showPassword, isSubmitting) for form UI state; useForm custom hook (values, errors, handleChange, handleSubmit, reset, isValid) to keep validation out of the screen; useAuth (useContext) to store the user.

7.2 Menu Browsing Screen

Purpose	Show the menu, filter by category and add items to the cart.
UI elements	Horizontal category chips, FlatList of MenuItemCards (image, name, price, Daily Special badge, favourite heart, Add button), loading indicator, error message with Retry, pull to refresh, header with item count.
Navigation (entry / exit)	Entry: Menu tab (default for Customer). Exit: Cart tab, Search controls on the same screen.
Data used	src/data/menu.js loaded through a 1.5 second Promise; favouriteIds array in state.
Hooks and why	useState (menuItems, isLoading, error, selectedCategory, refreshing, favouriteIds); useEffect with empty dependency array to load data with cleanup that clears the timer; useMemo for filtered and sorted list (derived data is not stored in state); useCallback + React.memo for MenuItemCard so only the toggled card re-renders.

7.3 Search and Scroll Controls

Purpose	Let the user find dishes fast and move around a long list.
UI elements	Search bar with search icon and clear button, recent-search suggestions (last five), sort selector, empty-state message, floating Back to top button, render counter debug label.
Navigation (entry / exit)	Entry: part of the Menu screen. Exit: none (same screen).
Data used	Menu items in state; recentSearches array in state.
Hooks and why	useRef for the TextInput (focus), the FlatList (scrollToOffset) and the render counter (changing a ref does not re-render, changing state does); useDebounce(value, 400) custom hook for the search text; useState for scroll position (button visible after 300 px) and recent searches; useMemo for filtering and sorting.

7.4 Profile and Theme Screen

Purpose	Show account details and let the user switch between light and dark theme or log out.
UI elements	Name, email and role fields, dark mode Switch, Logout button.
Navigation (entry / exit)	Entry: Profile tab. Exit: Login screen on logout (navigation stack reset).
Data used	AuthContext user; theme palettes in one theme file.
Hooks and why	useContext through useAuth and useTheme (both throw an error outside their provider) so every screen reads user and colours without prop drilling.

7.5 Cart Screen

Purpose	Review and change the cart, apply a promo code.
UI elements	List of cart items with quantity stepper, remove button, special-instructions TextInput, promo code field with Apply and Remove, subtotal, Place Order button, tab badge with item count.
Navigation (entry / exit)	Entry: Cart tab. Exit: Order Summary screen.
Data used	Cart state { items, promoCode, discountPercent }; local promo object (WELCOME10, FEAST20).
Hooks and why	useReducer with cartReducer (ADD_ITEM, REMOVE_ITEM, INCREMENT, DECREMENT, UPDATE_NOTE, CLEAR_CART, APPLY_PROMO, REMOVE_PROMO) because the cart has several related transitions; CartContext and useCart share it with the menu cards and tab badge.

7.6 Order Summary Screen

Purpose	Show the price breakdown and confirm the order.
UI elements	Line items, subtotal, service charge 5%, sales tax 15%, promo discount, grand total, Dine-in/Takeaway selector, table picker or pickup time, Confirm button.
Navigation (entry / exit)	Entry: from Cart. Exit: Order Tracking screen after the order is placed.
Data used	Cart items and discount; constants SERVICE_RATE = 0.05 and TAX_RATE = 0.15.
Hooks and why	useMemo for totals depending only on cart items and discount; useState for order type, table and pickup time; useContext (OrdersContext) to dispatch the new order.

7.7 Table Reservation Screen

Purpose	Book a table for a date, time and party size.
UI elements	Date picker, hourly time-slot chips (12:00 to 22:00, disabled when unavailable), party size, table choice, phone input, confirmation modal, My Reservations list with Cancel.
Navigation (entry / exit)	Entry: Reservations tab. Exit: stays on screen; cancel shows an Alert.
Data used	mockTables and mockReservations; reservations persisted in AsyncStorage.
Hooks and why	useReservation custom hook holds all business logic (selection state, availability check, createReservation, cancelReservation) so the screen has only UI code; useForm for validation; useState for modal visibility.

7.8 Order Tracking and Manager Dashboard

Purpose	Customer: watch the order progress. Manager: manage orders, reservations and the menu.
UI elements	Tracking: step progress indicator, elapsed time counter, status text. Dashboard: three tabs (Incoming Orders with status buttons, Reservations with Accept/Decline, Menu Management with add item, edit price, availability switch).
Navigation (entry / exit)	Tracking entry: after placing an order. Dashboard entry: Manager tab, visible only when user.role is manager.
Data used	OrdersContext, reservations and menu state shared between customer and manager screens and saved in AsyncStorage.
Hooks and why	useEffect with setInterval for the one second elapsed timer and the 10 second polling, both cleared in the cleanup function; useReducer in OrdersContext; useEffect + AsyncStorage for persistence; a loading screen until data is loaded.